

Введение

Многие задачи, возникающие в таких фундаментальных науках как физика, химия, молекулярная биология, а также во многих прикладных науках, сводятся к задачам непрерывной глобальной оптимизации.

Отличительной особенностью таких задач часто является нелинейность, недифференцируемость, многоэкстремальность (мульти-modalность), овражность (когда у функции в одном направлении нет изменений, а в другом есть), отсутствие аналитического описания (плохая формализованность), высокая вычислительная сложность, большая размерность пространства поиска, сложная область допустимых значений и т.д.

С самой общей точки зрения указанные особенности объясняют отсутствие универсального алгоритма их решения. Число таких алгоритмов оптимизации увеличивается, растёт количество их параллельных версий, ориентированных на различные классы параллельных вычислительных систем.

Для решения такого класса задач в 80-ых годах XX-го века интенсивно стали развиваться стохастические поисковые алгоритмы (в некоторых публикациях их также называют поведенческими, интеллектуальными, метаэвристическими, вдохновлёнными – или инспирированными – живой природой, роевыми, многоагентными).

Наиболее широко распространено понятие “популяционный алгоритм”. Популяционные алгоритмы предполагают одновременную обработку нескольких вариантов решения задачи оптимизации и представляют собой альтернативу классическим траекторным алгоритмам, в которых в области поиска эволюционирует только один кандидат на решение.

Задачи глобальной оптимизации можно разделить на два класса: детерминированные и стохастические. В первом случае оптимизируемая функция и функции, ограничивающие область решений задачи, являются детерминированными (точно определёнными), во втором случае одна или несколько указанных функций содержат случайные параметры.

Среди задач глобальной оптимизации выделяют статические, т.е. стационарные, и динамические. В статических задачах оптимизируемая функция и область её допустимых значений не меняются во времени, т.е. остаются неизменными положения в области поиска её локальных и глобальных экстремумов. В динамических задачах положения экстремумов могут меняться во времени и задача состоит в отслеживании движения экстремума.

Таким образом предметом рассмотрения являются детерминированные статические задачи глобальной безусловной (без ограничений на варьируемый параметр) и условной (с ограничением на это значение) непрерывной оптимизации.

По способу определения направления движения к экстремуму алгоритмы поисковой оптимизации делят на алгоритмы детерминированного (или регулярного) поиска (алгоритм градиентного спуска) и алгоритмы стохастического или случайного поиска.

Алгоритмы поисковой оптимизации разделяют ещё на два следующих класса: использующие как пробные, так и рабочие шаги алгоритма; алгоритмы, в которых эти шаги совмещены.

Все популяционные алгоритмы являются эвристическими, то есть для них сходимость к глобальному решению не доказано, но экспериментально установлено, что они дают достаточно хороший результат достаточно быстро.

В качестве общего названия для элементов популяции будем использовать термин агент или особь. Также возможны другие вариации (частица, индивид).

Общая схема всех популяционных алгоритмов включает следующую последовательность шагов:

1. Инициализация популяции. В области тем или иным образом создаётся некоторое число начальных приближений к искомому решению задачи, т.е. происходит процесс инициализации популяции агентов.
2. Миграция агентов. С помощью некоторого набора миграционных операторов, специфических для каждого популяционного алгоритма, агенты перемещаются в области поиска, чтобы приблизиться к искомому значению оптимизируемой функции.
3. Завершение поиска. Осуществляется проверка выполнения условий окончания итераций: если они выполнены – завершаем вычисления, принимаем лучшие значения агентов за приближённое решение задачи, иначе – переход к шагу 2.

При инициализации популяции можно использовать детерминированные или случайные алгоритмы. Формирование начальной популяции, агенты которой находятся вблизи глобального экстремума, может существенно сократить время решения задачи. Обычно априорная информация о местоположении экстремума отсутствует, поэтому агентов начальной популяции распределяют равномерно по всей области поиска.

Миграцию агентов, например, в генетическом алгоритме (ГА) реализуют генетические операторы (кроссовер, мутация).

В качестве условий окончания поиска, как правило, используется достижение заданного числа итераций или поколений. Также используется условие стагнации (проверка на изменения значений целевой функции: если изменений значения целевой функции практически нет, то останавливаем алгоритм).

Из представленной общей схемы следует, что популяционные алгоритмы обладают ярко выраженной модульной структурой, что позволяет легко получать большое число вариантов любого из алгоритмов путём варьирования и комбинации правил инициализации, миграции и завершения поиска.

Агенты популяции обладают следующим рядом свойств:

1. Автономность. Агенты движутся в пространстве хотя бы частично независимо друг от друга.
2. Стохастичность. Процесс миграции агентов всегда содержит случайную компоненту.

3. Ограниченность представления. Каждый агент обладает информацией только об исследуемой им части области поиска и возможных об окружении некоторых соседних агентов.

4. Децентрализация. Отсутствие агентов, управляющих процессом поиска.

5. Коммуникабельность.

Агенты тем или иным способом могут обмениваться информацией о топологии (ландшафте) оптимизируемой функции.

Даже если стратегия поведения каждого агента достаточно проста, указанные свойства обеспечивают формирование роевого интеллекта, определяющие организацию и сложное поведение популяции в целом.

Одной из особенностей популяционных алгоритмов является то, что в подавляющем большинстве случаев они имеют аналогии в человеческом обществе, живой и неживой природе: алгоритмы эволюции разума, колонии муравьёв, медоносных пчёл, сорной травы, гравитационного взаимодействия и т.д.

Популяционные алгоритмы в сравнении с классическими неоспоримые преимущества: прежде всего в задачах высокой размерности мультимодальных (мультиэкстремальных) и плохо формализуемых задачах. Важно также, что популяционные алгоритмы позволяют эффективнее классических искать субоптимальное решение (приблизительное).

Популяционные алгоритмы практически наверняка проиграют точному при решении задачи небольшой размерности. В случае гладкой мультимодальной функции популяционные алгоритмы менее эффективны, чем, например, градиентный спуск. К недостаткам также можно отнести зависимость от значений свободных параметров.

Важнейшим понятием популяционного алгоритма является fitness-функция (целевая функция, функция пригодности, функция приспособленности). Она используется для оценки качества агентов популяции. Часто, но не всегда fitness-функция совпадает с оптимизируемой функцией.

С используем fitness-функции связана суть популяционных алгоритмов, заключающаяся в обеспечении более высокой средней приспособленности популяции агентов данного поколения по сравнению с приспособленностью предыдущего поколения.

Одна из основных проблем конструирования популяционных алгоритмов – обеспечение баланса между интенсивностью (скоростью сходимости) и широтой поиска, т.е. диверсификацией.

Первые поколения популяционного алгоритма занимаются исследованием области поиска, агенты последних поколений заняты уточнением исключительно найденных решений. Баланса между интенсивностью и широтой можно добиться, используя механизмы адаптации и самоадаптации.

Основными критериями эффективности популяционных алгоритмов являются надёжность, т.е. вероятность локализации глобального экстремума; а также скорость сходимости, т.е. оценка матожидания необходимого числа испытаний.

Классификация популяционных алгоритмов:

1. ЭА (эволюционные алгоритмы), в том числе ГА (генетические алгоритмы).
2. Популяционные алгоритмы, вдохновлённые живой природой.
3. Алгоритмы, вдохновлённые неживой природой.
4. Алгоритмы, инспирированные человеческим обществом.
5. Прочие алгоритмы.

Постановка и классификация алгоритмов решения детерминированной задачи поисковой оптимизации

Рассмотрим детерминированную задачу оптимизации. Минимизировать функцию:

$$\min_{x \text{ belongs to } D} f(x) = f(x^*) = f^*$$

$$x = (x_1, \dots, x_{|x|})$$

$|x|$ — размерность по модулю варьируемых параметров

R — $|x|$ -мерное арифметическое пространство

$f(x)$ — целевая функция (критерий оптимальности)

x^*, f^* — искомое оптимальное решение и значение целевой функции

$D = \{x \mid E(x) = 0, G(x) \geq 0\}$ — область поиска (множество с ограничениями в виде равенств или неравенств)

Детерминированность поставленной задачи означает, что целевая и ограничивающая функции $E(x)$, $G(x)$ не содержат случайных параметров.

Целевую функцию $f(x)$, где x belongs to $[a; b]$ называется унимодальной, если в её области определения существует такая x^* , что на полуинтервале $[a; x^*)$ $f(x)$ убывает, а на $(x^*; b]$ $f(x)$ возрастает, при этом x^* может располагаться как внутри отрезка $[a; b]$, так и являться одним из его концов.

Непрерывную в своей области определения (на отрезке $[a; b]$) одномерную целевую функцию $f(x)$ называют выпуклой вниз, если для любых x_1, x_2 belongs to $[a; b] \Rightarrow f(Lx_1 + (1 - L)x_2) \leq Lf(x_1) + (1 - L)f(x_2)$

Определение выпуклой целевой функции не требует её унимодальности. Здесь x_1 и x_2 не равны, L belongs to $[0; 1]$.

Если заменить все нестрогие неравенства на строгие, то получим определение строго выпуклой функции. Строго выпуклая функция является унимодальной.

Целевую функцию, имеющую в своей области определения несколько локальных минимумов называют многоэкстремальной или мультимодальной.

Целевую функцию называют овражной к своей области определения, если в этой области имеют место слабые изменения первых частных производных функции по одним направлениям и значительные изменения по другим.

Если целевая функция представляет собой сумму нескольких функций, каждая из которых зависит только от одной компоненты вектора x , то такую функцию называют сепарабельной.

Функцию называют полиномиальной, если она представляет собой сумму произведений $c_i x^a$, где c_i , a — любые действительные числа.

Однопараметрические функции:

униmodalные, мультимодальные;
выпуклые, строго выпуклые.

Многопараметрические:

униmodalные, мультимодальные;
выпуклые, строго выпуклые;
овражные;
сепарабельные;
полиномиальные.

Классификация задач оптимизации

Рассмотрим основные признаки классификации оптимизационных задач:

1. Вид целевой и ограничивающих функций. Если целевая функция линейная, а множество допустимых значений D — выпуклый многогранник, то задачу называют задачей линейного программирования. Если $f(x)$ — отношение двух линейных функций, то задача носит название дробно-линейного программирования. Если область допустимых значений D определяется только ограничениями типа неравенств, а целевая и ограничивающая функции являются сепарабельными, то такую задачу называют задачей сепарабельного программирования. Если целевая и ограничивающая функции ($E(x)$, $G(x)$) являются полиномами — задача геометрического программирования. Если целевая функция является выпуклой, то и задача называется задачей выпуклого программирования. При квадратичной целевой функции и выпуклом множестве D задача называется задачей квадратичного программирования. Конечное множество D порождает задачу дискретного программирования, а множество D как множество целых чисел — задачу целочисленного программирования. В общем случае задачу минимизации называют задачей нелинейного программирования. Часто задачи выпуклого программирования также относят к задачам нелинейного программирования.

2. Наличие или отсутствие ограничений. Если ограничение на вектор x отсутствует ($D = R^{|x|}$), то задачу называют оптимизацией без ограничений или задачей безусловной оптимизации. При наличии ограничений на вектор x задачу называют задачей с ограничениями или условной оптимизацией (D lies in $R^{|x|}$).
3. Характер ограничений. $D = \{x \mid G(x) \geq 0\}$, то и задача носит название задачи с ограничениями типа неравенств. Если $D = \{x \mid E(x) = 0\}$, то задача носит название задачи с ограничениями типа равенств. Если $D = \{x \mid E(x) = 0, G(x) \geq 0\}$, то задача носит название задачи с ограничениями общего вида.
4. Размерность вектора x . Если размерность вектора x равна 1, то задача называется задачей однопараметрической оптимизации. Если размерность вектора x больше 1, то задача называется задачей многопараметрической оптимизации.
5. Число точек минимума. Если в области допустимых значений существует только один экстремум, то задачу называют одноэкстремальной. Если более одного — многоэкстремальной.
6. Характер искомого решения. Если отыскивается любой локальный минимум в области D , то задачу называют задачей локальной оптимизации. Если целью является поиск глобального минимума функции, то и задача называется задачей глобальной оптимизации.

Классификация алгоритмов оптимизации

Испытанием называют операцию однократного вычисления значений целевой функции $f(x)$ и ограничивающих функций, представленных в виде $G(x)$ и $E(x)$, а также, быть может, производных указанных функций в некоторой точке x из области допустимых значений. Особенностями задач оптимизации, представляющих практический интерес, является тот факт, что каждое испытание может требовать больших затрат компьютерного времени, поэтому одним из основных требований, предъявляемых к алгоритмам оптимизации, является решение задачи оптимизации при наименьшем числе испытаний. Приближённое решение задачи или кандидата на решение будем называть агентом и обозначать как S_i i belongs to $[1; |S|]$, $|S| \geq 1$, где S — используемое множество агентов (популяция агентов). Будем рассматривать итерационные алгоритмы, которые наиболее распространены в вычислительной практике. Рассмотрим общую схему итерационного алгоритма:

1. Инициализируем алгоритм. Задаём начальное значение счётчика числа итераций $t=0$, начальное положение агента $x_i(0)$, а также значение свободных параметров.
2. Применяем поисковые (миграционные) операторы $x_i(t)=x_i^t=x_i$ и получаем новые положения $x_i(t+1)$.
3. Проверяем выполнение условий завершения итераций. Если они не выполнены, то $t+=1$ и возврат к шагу 2. В противном случае принимаем лучшее из найденных положений агента за приближённое решение задачи.

С самой общей точки зрения миграционные операторы вычисляют новое положение агента и проверяют на соответствие функции, зависящей от набора значений: x_k^t , $f(x_k^t)$, $E(x_k^t)$, $G(x_k^t)$, k belongs to $[1; |S|]$, t belongs to $[0; t']$. Общий смысл состоит в том, что новое положение агента S_i вообще говоря определяется всеми предыдущими положениями всех агентов популяции, а также соответствующими значениями целевой и ограничивающих функций. Здесь t' есть число итераций. В качестве приближённого решения задачи принимается $f^*=f(x^*)=\min f(x_k^t)$ t belongs to $[0; t']$. f^* — минимальное значение функции среди всех значений агентов.

В итерационных алгоритмах $F(x_k^t)$, $f(x_k^t)$, $E(x_k^t)$, $G(x_k^t)$) не меняется с течением числа итераций, за исключением, возможно, некоторого числа начальных шагов.

Поисковые алгоритмы оптимизации можно классифицировать в соответствии со следующими признаками:

1. Характер искомого решения. Алгоритмы называют локальными, если схема поиска нацелена на отыскание локального минимума. Если целью является поиск глобального минимума, то алгоритм будет глобальным.
2. Характер ограничений. Алгоритм называется алгоритмом безусловной оптимизации, если ориентирован на решение соответствующей задачи — задачи безусловной оптимизации. Алгоритм называется алгоритмом условной оптимизации, если ориентирован на решение задачи условной оптимизации.
3. Характер функции F . Если F является детерминированной, то и алгоритм называют детерминированным. Если F содержит один или несколько параметров, которые на различных итерациях принимают случайные значения с заданными законами распределения, то алгоритм называется стохастическим (случайным).
4. Класс алгоритма. Алгоритм носит название пассивного, если все точки x_i^t назначены заранее. Если же данные точки определяются на основе всей или части информации об испытаниях предыдущих точек, то алгоритм называют последовательным.
5. Число предыдущих учитываемых шагов. Если при вычислении новых координат точки учитывается информация об одной итерации (предыдущей), то алгоритм называется одношаговым. Если при вычислении новых координат точки учитывается информация о более чем одном предыдущем испытании, то алгоритм называется многошаговым (с памятью).
6. Порядок используемых производных. Алгоритмы поиска называют прямыми или алгоритмами нулевого порядка, если при вычислении целевой функции и ограничивающих функций не используются производные. Если в тех же условиях используются производные k -ого порядка, то алгоритм называется алгоритмом k -ого порядка.

Выделяют также траекторные и популяционные алгоритмы оптимизации. В траекторных алгоритмах на каждой итерации обновляется положение только одного агента. При этом общее число агентов, как правило, больше одного и на разных итерациях перемещаются разные агенты. Траекторные алгоритмы могут быть одноточечными и многоточечными. В популяционных алгоритмах число агентов всегда больше одного и на каждой итерации перемещаются все агенты.

Важной проблемой при построении поисковых алгоритмов оптимизации является проблема выбора условий или критериев окончания поиска. Простейшими, но широко используемыми в вычислительной практике являются два следующих условия: $\|x^{t+1} - x^t\| \leq e_x$ и $|f(x^{t+1}) - f(x^t)| \leq e_f$. Данные условия останова поиска называются стандартными.

Метод поиска с использованием производных. Алгоритм градиентного спуска с постоянным шагом

Пусть дана функция $f(x)$, ограниченная снизу в n -мерном пространстве поиска и имеющая непрерывные частные производные во всех её точках. Требуется найти локальный минимум функции $f(x)$ на множестве допустимых значений D и найти такую точку x^* , чтобы она соответствовала минимуму функции.

Градиентные методы или методы первого порядка используют значения частных производных первого порядка. Стратегия решения задачи состоит в построении последовательности точек, таких, что: $f(x^{k+1}) < f(x^k)$. При этом $x^{k+1} = x^k - t_k \text{grad } f(x^k)$.

Величина шага t_k остаётся постоянной до тех пор, пока функция убывает в точках последовательности. Проверяется это выполнением условия: $|f(x^{k+1}) - f(x^k)| < \epsilon * \|\text{grad } f(x^k)\|^2 \Leftrightarrow f(x^{k+1}) < f(x^k)$

Построение точек x^k заканчивается в той точке, для которой выполняется условие точности: $x^k : \|\text{grad } f(x^k)\| < \epsilon_1$ либо по числу итераций $k \geq M$, где k – число итераций, M – предельная величина. Либо при одновременном выполнении: $\|x^{k+1} - x^k\| < \epsilon_2$ и $|f(x^{k+1}) - f(x^k)| < \epsilon_2$

Алгоритм:

Шаг 1. Задаём входные данные, то есть x , $0 < \epsilon < 1$, ϵ_1 , ϵ_2 , M

Находим $\nabla f(x) = (f'_{x_1}(x), \dots, f'_{x_n}(x))^T$

Шаг 2. $k = 0$

Шаг 3. Вычисляем $\|\nabla f(x^k)\|$

Шаг 4. Проверяем выполнение $\|\nabla f(x^k)\| < \epsilon_1$ — критерий останова

Если критерий выполнен, то расчёт закончен, принимаем текущую точку как оптимальную

Если критерий не выполнен, то переходим к шагу 5

Шаг 5. Проверка условия $k \geq M$ — условие окончания по числу итераций

Если условие выполняется, то найдена исходная точка оптимума $x^* = x^k$

В противном случае переходим к шагу 6

Шаг 6. Задаём t_k — шаг спуска

Шаг 7. Вычисление следующей точки $x^{k+1} = x^k - t_k \nabla f(x^k)$

Шаг 8. Проверка третьего условия останова

Если условие выполнено, то переходим к шагу 9

В противном случае величина шага t_k уменьшается вдвое и переходим к шагу 7

Шаг 9. $x^* = x^{k+1}$, x^* — найденное оптимальное решение

Понятие алгоритма симплекс-метода

Симплекс-метод — это метод последовательного перехода от одного базисного решения системы ограничений задачи линейного программирования к другому до тех пор, пока функция цели не примет оптимальное значение. Симплекс-метод является универсальным и способен решить любую задачу линейного программирования (любой размерности).

Всякое неотрицательное решение системы ограничений называется допустимым решением. Пусть имеется система M ограничений с N переменными, где $M < N$. Допустим, что базисным решением является решение, содержащее M неотрицательных основных, то есть базисных переменных и $N - M$ неосновных, то есть не базисных или несвободных переменных. Неосновные переменные в базисном решении равны нулю, основные, как правило, отличны от нуля, то есть являются положительными числами. Любые M переменных системы M линейных уравнений с N переменными называют основными, если определитель, составленный из коэффициентов при них отличен от нуля. Тогда остальные $N - M$ называют неосновными или свободными.

Алгоритм симплекс-метода:

Шаг 1. Привести задачу линейного программирования к канонической форме. Для этого перенести свободные члены в правые части. Если при этом среди них окажутся отрицательные, то соответствующие уравнение или неравенства нужно умножить на -1 . В каждое ограничение ввести дополнительные переменные со знаком $+$, если в исходном уравнении стоял знак $<$ и со знаком $-$, в случае, если $\geq$

Шаг 2. Если в полученной системе M уравнений, то M переменных принимают за основные, после чего нужно выразить основные переменные через неосновные и найти соответствующее базисное решение. Если найденное базисное решение окажется допустимым, то нужно принять данное решение.

Шаг 3. Выразить функцию цели через неосновные переменные допустимого базисного решения. Если отыскивается максимум (минимум) линейной формы и в её выражении нет неосновных переменных с отрицательными (положительными) коэффициентами, то критерий оптимальности выполнен и полученное решение является оптимальным. Если при нахождении максимума (минимума) линейной формы в её выражении имеется одна или несколько переменных с отрицательными (положительными) коэффициентами, то осуществляется переход к новому базисному решению.

Шаг 4. Из неосновных переменных, входящих в линейную форму с неотрицательными (положительными коэффициентами), выбирают ту, которой соответствует наибольший по модулю коэффициент и переводят её в основные. Далее переход к шагу 2.

Если допустимое решение даёт оптимум линейной формы, то есть выполняется критерий оптимальности, а в выражении линейной формы через неосновные переменные отсутствует хотя бы одна из них, значит, полученное оптимальное решение не единственно.

Если в выражении линейной формы имеется неосновная переменная с отрицательным коэффициентом в случае максимизации и положительным в случае минимизации, а в остальные уравнения системы ограничений этого шага эта переменная также входит с отрицательными коэффициентами или отсутствует, то линейная форма неограничена при данной системе ограничений.

Пример:

1	2	3
$F = x_1 + 2x_2$ $-x_1 + 2x_2 \geq 2$ $x_1 + x_2 \geq 4$ $x_1 - x_2 \leq 2$ $x_2 \leq 6$	$-x_1 + 2x_2 - x_3 = 2$ $x_1 + x_2 - x_4 = 4$ $x_1 - x_2 + x_5 = 2$ $x_2 + x_6 = 6$	$F = 0 - (-x_1 - 2x_2)$ $x_3 = -2 - (x_1 - 2x_2)$ $x_4 = -4 - (-x_1 - x_2)$ $x_5 = 2 - (x_1 - x_2)$ $x_6 = 6 - x_2$

Таблица 1

Таблица 1				
Базисные неиз- вестные	Свободные чле- ны	Свободные неизвестные		Вспомогательные коэффициенты
		x1	x2	
x3	-2	1	-2 – ведущий	
x4	-4	-1	-1	
x5	2	1	-1	
x6	6	0	1	
F	0	-1	-2	

Для перехода к следующей симплекс-таблице находим наибольшие по модулю из чисел из свободных неизвестных. Это ведущий столбец. Для определения ведущей строки находим минимум из соотношений свободных членов к элементам ведущего столбца. Новый базисный элемент x_2 вписываем первой строкой, а в столбец, в котором стояла x_2 , записываем новую свободную переменную x_3 . Заполняем вторую симплекс-таблицу.

Таблица 2

Таблица 2				
Базисные неиз- вестные	Свободные чле- ны	Свободные неизвестные		Вспомогательные коэффициенты
		x1	x2	
x3	1	-1/2	-1/2	
x4	-3	-3/2	-1/2	1
x5	3	1/2	-1/2	1
x6	5	1/2	1/2	-1
F	2	-2	-1	2

Заполняем первую строку. Для этого все числа, стоящие в ведущей строке первой таблицы делим на ведущий элемент. И записываем в соответствующий столбец первой строки второй таблицы, кроме ячейки с числом, стоящим на пересечении ведущего строки и столбца. Туда записываем обратное к нему. В столбец вспомогательных коэффициентов...

...

$$x_1 = 8, x_2 = 6, F_{\max} = 8 + 12 = 20$$

[simplex-method-full-explaining](#)

Эволюционные алгоритмы

Эволюционные алгоритмы включают в себя генетические алгоритмы, эволюционную стратегию, эволюционное программирование, а также генетическое программирование. Суть парадигмы эволюционных алгоритмов состоит в использовании базовых принципов теории биологической эволюции, то есть отбора, мутации и воспроизведения особей. Эволюционные алгоритмы являются частью более широкой технологии так называемых мягких вычислений, включающих в себя нечёткую логику, нейронные сети, вероятностные рассуждения, а также сети доверия. Наиболее развитый класс эволюционных алгоритмов — это ГА (генетические алгоритмы).

Биологические предпосылки и общая схема эволюционных алгоритмов.

Чарльз Дарвин, Жан Батист Ламарк, Гюго Де Фриз

Доступно чрезвычайно большое число источников, посвящённых различным аспектам эволюционных алгоритмов. Наиболее популярна работа Чарльза Дарвина «Происхождение видов». Движущей силой эволюции по Дарвину являются наследственность, изменчивость и естественный отбор.

В каждой клетке живого организма содержится информация о нём в ДНК.

Структуру, содержащую ДНК, называют хромосомой.

Ген — часть хромосомы, кодирующая определённые свойства особи.

Совокупность генов образует генотип, на основе которого формируется фенотип, то есть совокупность характеристик, присущих особи на определённой стадии её развития.

Локус — позиция гена в хромосоме.

Аллель — координата точки.

Взаимодействие ДНК — скрещивание (кроссовер или рекомбинация).

В результате внешних факторов могут происходить случайные ненаправленные изменения — мутация.

Таким образом, можно сформировать схему классического генетического алгоритма:

1. Агентов представляем в виде хромосом.
2. Случайным образом создаём некоторое число исходных особей — начальную популяцию.
3. Оцениваем особей с помощью целевой функции. Каждой особи ставится в соответствие определённое значение, которое определяет вероятность его выживания.
4. На основе приспособленности выбираем особей для скрещивания. К хромосомам этих особей применяем генетические операторы (кроссовер, мутация, транспозиция, транслокация, инверсия, усечение и т.д.), создавая следующее поколение особей.
5. Особей созданного поколения также оцениваем.
6. Повторяем 3, 4, 5 до достижения критерия остановки.

Известно большое число вариантов рассмотренной схемы ГА, отличающееся способами кодирования хромосом, набором генетических операторов, структурой и параметрами алгоритма.

Основными являются следующие генетические операторы:

1. Оператор мутации, реализующий случайное изменение одного или нескольких генов в хромосоме.
2. Оператор кроссовера (скрещивания), предназначенный для создания особей потомков путём рекомбинации хромосом двух и более родителей.
3. Оператор управления популяцией, формирующий на основе популяции текущего поколения T промежуточную популяцию S' .
4. Оператор селекции, осуществляющий выбор из промежуточной популяции S' особей для скрещивания и формирующий популяцию следующего поколения $T + 1$.

В простейшем случае промежуточная и текущая популяции совпадают.

Генетические операторы используют некоторое число свободных параметров, которые могут меняться в процессе вычислений в зависимости от числа поколений либо адаптивно зависеть от эффективности функции. Такие операторы называют неравномерными и адаптивными соответственно. Для эволюционных алгоритмов и для ГА в частности остро стоит вопрос сходимости.

Пусть последовательность популяций $S(t)$, где t от 0 до числа поколений, монотонна, т.е. при поиске максимума не убывает и при поиске минимума не возрастает. Пусть также для любых двух особей возможен переход от одной из них ко второй посредством однократного или многократного применения операторов мутации и кроссовера. Тогда ГА отыщет глобально оптимальную особь, доставляющую максимум функции с вероятностью равной 1.

Кодирование особей

Чаще всего используют бинарное либо вещественное кодирование генов в хромосоме. А также большое число способов кодирования, приводящих к следующим основным типам особей:

1. Монохромосомные особи — это классический способ кодирования значений варьируемых параметров в виде бинарной строки фиксированного вида.
2. Мультихромосомные особи — каждая часть мультихромосомы отвечает за определённый аспект решений.
3. Адаптивные особи — предполагают, что в процессе поиска хромосома меняет свою структуру и длину.
4. Символьное кодирование — представлений хромосомы в виде последовательности символов. При высокой размерности вектора варьируемых подпараметров и высокой точности число бит особи в хромосоме оказывается чрезвычайно большим.

Вещественное кодирование

При решении задач оптимизации в непрерывных пространствах бинарное и символьное кодирование являются весьма ограниченными, поэтому разработано вещественное кодирование, когда гены в хромосоме напрямую представляются в виде вещественных чисел, точность которых ограничивается только ресурсами компьютера. Такое кодирование требует применения специальных генетических операторов. Алгоритм, использующий непрерывное кодирование, называют непрерывным ГА (Real Coded GA). По аналогии с этим алгоритмы, использующие бинарное кодирование, называют бинарными ГА (Binary Coded GA).

Только оператор отбора пригоден как для бинарного, так и для непрерывного ГА. Операторы скрещивания и мутации, используемые в классических эволюционных алгоритмах, не позволяют работать с вещественными хромосомами.

Математический аппарат непрерывных ГА

При работе в непрерывных пространствах логично представлять гены напрямую вещественными числами. Длина хромосомы будет совпадать с длиной вектора решения оптимизационной задачи, т.е. каждый ген отвечает за одну переменную, а генотип объекта становится идентичным его фенотипу. Всё вышесказанное определяет список основных преимуществ непрерывных ГА:

1. Использование непрерывных генов делает возможным поиск в больших пространствах, что является затруднительным в случае бинарного кодирования, где увеличение пространства поиска уменьшает точность решения при неизменной длине хромосом.
2. Локальная настройка решений.
3. Такой способ представления решений близок к постановке большинства прикладных задач, а отсутствие операций кодирования и декодирования, присущих БГА, повышает скорость работы.

Операторы отбора

Выделяют два близких, но функционально отличающихся класса операторов отбора:

1. Операторы управления популяцией.
2. Операторы селекции.

Операторы управления популяцией делят на учитывающие только приспособленность особей (1.1) и учитывающие близость генотипов (1.2).

1.1.1. Метод рулетки или метод пропорционального отбора является простейшим методом отбора. Вероятность отбора особи S_i определяет $K_i = f(S_i) / \sum_{j=1}^{|S|} f(S_j)$. Схему отбора метода рулетки можно представить в виде следующей последовательности:

- 1) для каждой особи S_i текущей популяции вычисляется её приспособленность;
- 2) генерируем случайное число в интервале от 0 до $2\pi i$;

- 3) помещаем особь S_i , соответствующую углу в промежуточную популяцию;
- 4) если требуемое число особей сформировано, то заканчиваем, иначе переходим к шагу 2.

1.1.2. Развитием метода рулетки является метод пропорционального отбора с остатком, который основывается на вычислении отношения значения целевой функции текущего решения к средней приспособленности (среднего значения целевой функции) популяции. Среднее значение указывает, сколько раз нужно записать особь в промежуточную популяцию, а дробная часть показывает вероятность попасть туда ещё раз.

1.1.3. Метод стохастического универсального отбора. Особи с самой высокой приспособленностью гарантированно выбираются хотя бы один раз.

1.1.4. Метод турнирного отбора. Метод предполагает случайное формирование на основе текущей популяции некоторого числа групп из N особей. В каждой из групп выбирается особь с наилучшей приспособленностью, которая включается в промежуточную популяцию. Величина турнира (размер) определяет давление селекции.

1.1.5. Метод рангового отбора. Его суть заключается в том, что число копий данной особи, включённых в промежуточную популяцию, является некоторой априори заданной величиной. Рангом называется номер этой особи в списке, отсортированном по возрастанию приспособленности.

1.1.6. Метод отбора на основе элитизма. Основан на предварительном выборе лучших, то есть наиболее приспособленных особей. Модификацией является метод отбора отсечением, когда задаётся величина порога приспособленности. Особей упорядочивают по значению функции. В промежуточную популяцию попадают особи, значения функции которых выше некоторого порога.

1.2.1. Метод отбора вытеснением. Основан на удалении из популяции в некотором смысле близких особей. $p(s_1, s_2)$ — величина близости двух особей. Схема метода отбора вытеснением: для популяции $S = \{s_1, \dots, s_{|S|}\}$ формируется матрица расстояний между особями популяции $R = (r_{ij})$. Далее выбирается элемент матрицы R_{ij} , имеющий минимальное значение, и с равной вероятностью из популяции удаляется либо s_i , либо s_j и т.д.

2. Как правило, селекцию выполняют, оценивая приспособленность особи. Используется следующий принцип: вероятность отбора наименее приспособленных особей должна уменьшаться. Методы селекции можно разделить на три класса:

2.1. Случайные методы, не учитывающий приспособленность и генотип особей.

2.2. Методы, использующие только приспособленность.

2.3. Методы, основанные только на генотипе.

2.1.1. Метод панмиксии. Это случайный равновероятный выбор особей из промежуточной популяции. Схема следующая: каждой особи в популяции ставится в соответствие значение вероятности $1/|S|$, где S — популяция. Далее методом рулетки выбираем из популяции первую особь и аналогичным образом выбираем остальные.

2.2.1. Метод селективного отбора. Выбираются только те особи, приспособленность которых не ниже средней приспособленности популяции. При этом полагают, что все особи, удовлетворяющие данному условию, могут быть выбраны с равной вероятностью. Данный метод

обеспечивает высокую скорость сходимости, но не является эффективным на многоэкстремальных функциях.

2.3.1. Инбридинг. Метод селекции, в котором первая особь выбирается случайно, а в качестве второй с большей вероятностью будет выбрана в некотором смысле наиболее близкая по генотипу особь. В качестве меры близости используется $p(s_i, s_j)$. То есть инбридинг — близкородственное скрещивание. Данный метод часто приводит к разбиению популяции на отдельные группы особей, локализующихся вблизи точек оптимальных решений.

2.3.2. Аутбридинг. Предполагает выбор первой особи по схеме панмиксии, а в качестве второй с большей вероятностью выбирают особь, которая в смысле близости генотипов $p(s_i, s_j)$ наиболее далека от первой. Аутбридинг предупреждает раннюю сходимость, т.е. обеспечивает высокую диверсификацию поиска (поиск более разнообразен).

Операторы скрещивания в непрерывном ГА

Требуется из двух векторов вещественных чисел получить новые векторы по каким-либо законам, но как правило, большинство ГА получают потомков в окрестности уже имеющихся особей. Рассмотрим наиболее известные схемы работы операторов кроссовера для непрерывных ГА.

Пусть $c_1 = (c_{11}, c_{12}, \dots, c_{1n})$ и $c_2 = (c_{21}, c_{22}, \dots, c_{2n})$ — две хромосомы, выбранные оператором селекции для скрещивания.

1. Простейший кроссовер случайным образом выбирает число $1 \leq k \leq n-1$ и генерирует на его основе двух потомков: $h_1 = (c_{11}, c_{12}, \dots, c_{1k}, c_{2(k+1)}, \dots, c_{2n})$ и $h_2 = (c_{21}, c_{22}, \dots, c_{2k}, c_{1(k+1)}, c_{1n})$.

2. Арифметический кроссовер. Создаются два потомка $h_1 = (h_{11}, \dots, h_{1n})$ и $h_2 = (h_{21}, \dots, h_{2n})$. Здесь $h_{1k} = w * c_{1k} + (1-w) * c_{2k}$ и $h_{2k} = w * c_{2k} + (1-w) * c_{1k}$, w belongs to $[0;1]$.

3. Геометрический кроссовер. Создаёт два потомка $h_1 = (h_{11}, \dots, h_{1n})$ и $h_2 = (h_{21}, \dots, h_{2n})$. Здесь $h_{1k} = c_{1k} ** w * c_{2k} ** (1-w)$ и $h_{2k} = c_{2k} ** w * c_{1k} ** (1-w)$.

4. Линейный кроссовер. Создаётся три потомка $h_q = (h_{q1}, \dots, h_{qn})$. $h_{1k} = 0,5 * c_{1k} + 0,5 * c_{2k}$; $h_{2k} = 1,5 * c_{1k} - 0,5 * c_{2k}$; $h_{3k} = -0,5 * c_{1k} + 1,5 * c_{2k}$.

5. Дискретный кроссовер. Каждый ген (h_k) выбирается случайно по равномерному закону из конечного множества $\{c_{k1}, c_{k2}\}$.

6. Расширенный линейчатый кроссовер. Ген (h_k) вычисляется так: $h_k = w * (c_{1k} - c_{2k}) + c_{1k}$, w belongs to $[-0,25; 1,25]$.

7. Эвристический кроссовер. —//— (то же самое, что и в р.л.к.) w belongs to $[0;1]$.

Операторы мутации

Случайные мутации. Как правило, наиболее часто реализуются. При случайной мутации ген, подлежащий изменению, принимает случайное значение из интервала своего изменения.

В неравномерной мутации значение гена рассчитывается как $c^*k = ck + \text{delta}$, if $w = 0$ или $c^*k = ck - \text{delta}$, if $w = 1$.

Сложно выделить наиболее эффективный оператор кроссовера или мутации, но экспериментально установлено, что применение непрерывных ГА показывает превосходство по сравнению с binary-coded ГА для задач оптимизации в многомерных пространствах. При этом real-coded ГА более просты в реализации за счёт отсутствия процедур кодирования и декодирования.

Алгоритм роя частиц

Алгоритм роя частиц был предложен в 1995 году. Идея алгоритма была частично заимствована из исследований поведения скоплений животных (косяков рыб, стай птиц и т.п.), модель была немного упрощена и добавлены элементы поведения толпы людей, поэтому, в отличие, например, от алгоритма пчел агенты алгоритма (возможные решения) и были названы нейтрально - частицы. Чтобы понять алгоритм роя частиц, представьте себе n -мерное пространство (область поиска), в котором рыщут частицы (агенты алгоритма). В начале частицы разбросаны случайным образом по всей области поиска и каждая частица имеет случайный вектор скорости. В каждой точке, где побывала частица, рассчитывается значение целевой функции. При этом каждая частица запоминает, какое (и где) лучшее значение целевой функции она лично нашла, а также каждая частица знает где расположена точка, являющаяся лучшей среди всех точек, которые разведали частицы. На каждой итерации частицы корректируют свою скорость (модуль и направление), чтобы с одной стороны быть поближе к лучшей точке, которую частица нашла сама (авторы алгоритма называли этот аспект поведения "ностальгией"), и, в то же время, приблизиться к точке, которая в данный момент является глобально лучшей. Через некоторое количество итераций частицы должны собраться вблизи наиболее хорошей точки, хотя возможно, что часть частиц останется где-то в относительно неплохом локальном экстремуме, но главное, чтобы хотя бы одна частица оказалась вблизи глобального экстремума. Самое интересное в алгоритме - это коррекция скорости, именно от этого шага зависит сходимость алгоритма. В первоначальном виде алгоритма коррекция скорости выглядела следующим образом:

$$v_{i,t+1} = v_{i,t} + \varphi_r p_i - x_{i,t} + \varphi_g g_i - x_{i,t}$$

Здесь $v_{i,t}$ – i -я компонента скорости при t -ой итерации алгоритма $x_{i,t}$ – i -я координата частицы при t -ой итерации алгоритма p_i – i -я координата лучшего решения, найденного частицей g_i – i -я координата лучшего решения, найденного всеми частицами φ_r, φ_g – случайные числа в интервале (0, 1) φ_r, φ_g – весовые коэффициенты, которые надо подбирать под конкретную задачу. Затем корректируем текущую координату каждой частицы

$$x_{i,t+1} = x_{i,t} + v_{i,t+1}$$

После этого рассчитываем значение целевой функции в каждой новой точке, каждая частица проверяет, не стала ли новая координата лучшей среди всех точек, где она побывала. Затем среди всех новых точек проверяем, не нашли ли мы новую глобально лучшую точку, и, если нашли, запоминаем ее координаты и значение целевой функции в ней. Формула для коррекции скорости частиц является сутью алгоритма, коэффициентов φ_r и φ_g были направлены

многие исследования, также были модифицированные алгоритмы роя частиц, коротко рассмотрим некоторые из них.

Учет инерции

Одна из модификаций алгоритма состоит в том, чтобы добавить еще один весовой коэффициент $\omega(t)$ перед текущей скоростью (коэффициент инерции), благодаря которому скорость изменяется более плавно.

$$v_{i,t+1} = \omega(t) v_{i,t} + \varphi_r r_p(p_i - x_i, t) + \varphi_g r_g(g_i - x_i, t)$$

Этот коэффициент может быть константой, а может зависеть от номера итерации t , например, линейно уменьшаться, начиная от небольшой величины, меньшей 1, и до какой-то другой величины, отличной от нуля.

Канонический алгоритм

Один из самых распространенных вариантов алгоритма вводит нормировку коэффициентов φ_r и φ_g , чтобы сходимость не так сильно зависела от их выбора.

$$v_{i,t+1} = \chi [v_{i,t} + \varphi_r r_p(p_i - x_i, t) + \varphi_g r_g(g_i - x_i, t)],$$

$$\varphi = \varphi_r + \varphi_g,$$

коэффициент k должен лежать в интервале $(0, 1)$.

Существуют и другие модификации алгоритма, которые, например, учитывают не точку, которую каждая частица запомнила, как лучшую, а точку, которая стала лучшей для самой частицы и ближайших ее соседей.

Алгоритм оптимизации пчелиным роем

Алгоритм пчелиной оптимизации был предложен в 2005 году. Основная идея — моделирование поведения пчёл при поиске нектара. Алгоритмы пчелиной оптимизации успешно применяются при решении задач дискретной оптимизации. Существует также высокоэффективный вариант алгоритма для решения задач непрерывной оптимизации. Среди них выделяют пчелиный алгоритм (В-алгоритм) и ABC-алгоритм, где А — artificial, В — bee, С — column. Основным недостатком пчелиных алгоритмов является большое число свободных параметров.

Введём следующую терминологию:

Источник нектара (источник пищи) — fitness-функция. Характеризуется своей полезностью, которая определяется на основе совокупности факторов: удалённость от улья, концентрация нектара, удобство добычи.

Рабочие пчёлы (занятые фуражиры, worker bees, W) — пчёлы, которые связаны с одним из источников нектара, т.е. добывают нектар на источнике. Занятые фуражиры владеют следующей информацией о своём источнике: направление от улья на источник и его полезность.

Незанятые фуражиры (пчёлы-разведчики, пчёлы-скауты, scout-bees) — осуществляют поиск источников нектара для их использования.

Незанятые пчёлы (looker bees, пчёлы-наблюдатели).

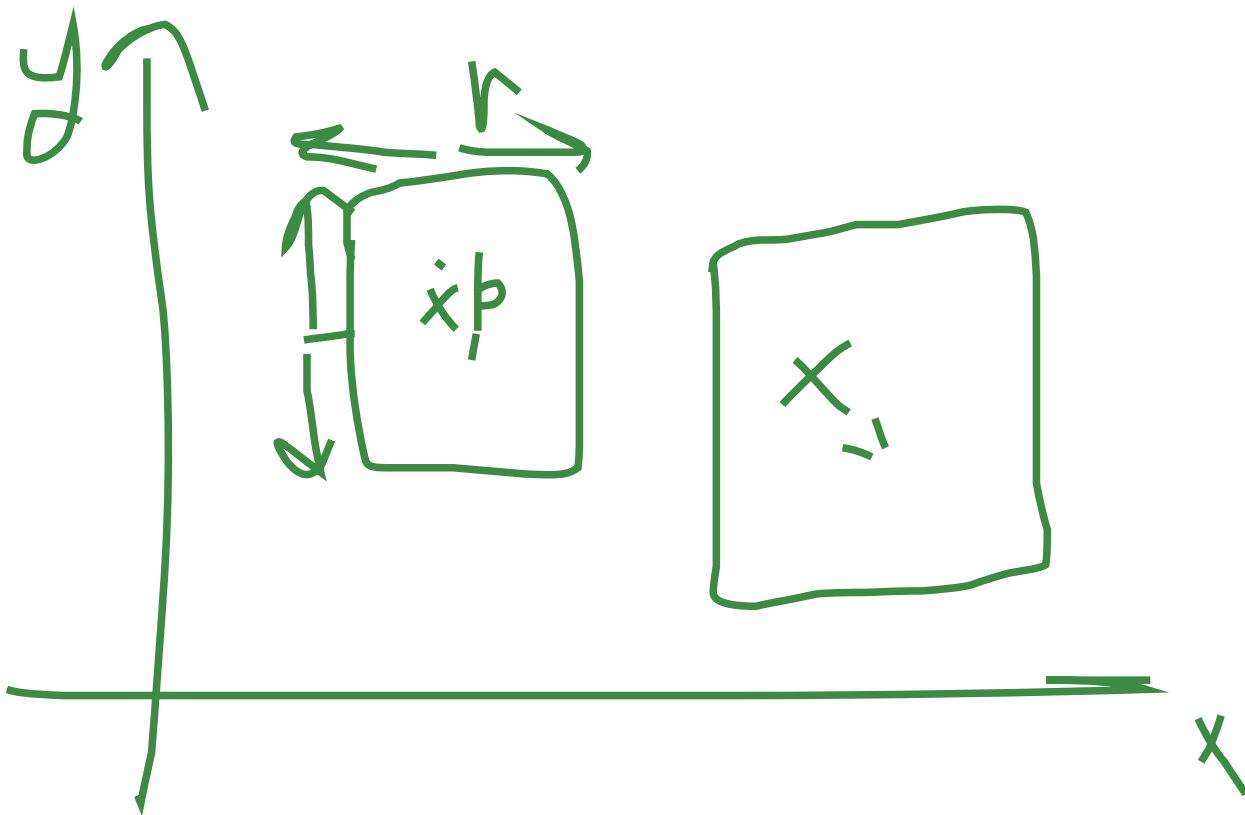
Самоорганизация пчелиного роя основывается на четырёх механизмах: положительная обратная связь на основе информации, полученной от разведчиков, пчела летит к одному из источ-

ников нектара; отрицательная обратная связь — пчела может решить, что её источник хуже других и оставить его; случайность — вероятность поиска нектара; множественность взаимодействия — информация об источнике нектара, найденном одной пчелой, передаётся многим.

Представим одну итерацию алгоритма в виде следующей последовательности шагов:

1. В случайные точки пространства поиска отправляем пчёл-разведчиков.
2. На основании значений целевой функции выделяем некоторое число элитных участков. Т.е. подобластей пространства поиска, соответствующих максимальным значениям целевой функции. Здесь же определяем некоторое число перспективных участков. Перспективные участки содержат точки, в которых значения близки к оптимальным.
3. Отправляем некоторое число рабочих пчёл на найденные участки. Находим новые элитные и перспективные участки. При выборе учитываем результаты, полученные как разведчиками, так и рабочими пчёлами. В качестве текущего приближения принимаем точку с оптимальным значением целевой функции. При этом размер элитных и перспективных участков уменьшается с ростом числа итераций.

Рассмотрим задачу глобальной условной максимизации в некотором пространстве, где $X \times Y$ — плоскость, Z — значения функции. $S = \{s_i\}$, S^o lies in S . S — весь рой. S^o — наблюдатели. $S - S^o = S^w$ — рабочие пчёлы. Значения целевой функции, соответствующей $s_i \rightarrow p_i$. A^b — множество элитных участков. A^p — множество перспективных участков. $A = A^b + A^p$ — отобранные участки.



Предполагаем, что участки имеют грани, параллельные координатным осям с центрами в точках C_i^j . Длины граней равны a_j . r^{ji} — радиусы. При формировании элитных и перспективных участков может оказаться так, расстояние между их центрами может оказаться меньше некоторой заранее заданной величины ϵ . В этом случае возможны несколько вариантов действий: поставить в соответствие обеим точкам один и тот же участок, центр которого располагается либо в первой, либо во второй области, но при этом учитывается, что значение функции в этой точке должно быть максимальным; двум различным точкам поставить в соответствие два различных, но при этом пересекающихся участка.

Во введенных обозначениях алгоритм выглядит так:

1. Генерируем x_i , равномерно распределённые в области поиска, отправляем в эти точки $|S^0|$ пчёл-разведчиков. Тут же вычисляем значение целевой функции и сортируем по убыванию.
2. Объявляем центры элитных и перспективных участков x_j и x_k соответственно.
3. В каждый из элитных и перспективных участков отправляем $n_b * A^b$ и $n_p * A^p$ пчёл соответственно. Отправленные пчёлы равномерно распределяются по участкам.
4. Проверяем условие окончания итерации. Выполнено — конец, иначе — переход к шагу 2.

Число рабочих пчёл, число элитных и перспективных участков, число пчёл, посылаемых на участки, размеры участков — гиперпараметры.

Рассмотренная схема В-алгоритма не учитывает при размещении пчёл-разведчиков их локализацию вблизи области поиска. Некоторые фрагменты элитных и перспективных участков могут выходить за границы области. Существует несколько вариантов решения этой проблемы: введение дополнительных функций штрафа; когда участки беспрепятственно выходят за границы области поиска, значения целевой функции этих пчёл игнорируются.

Пчелиный алгоритм — алгоритм с вещественным кодированием бессмертных особей. Процесс изменения координат пчёл можно интерпретировать как своеобразную мутацию.

Artificial Immune System

Первые исследования искусственных иммунных систем относят к 70-м, однако представляющие практический интерес результаты были получены в 90-х. ИИС — это информационная технология, использующая понятия «аппарат» и некоторые результаты теоретической иммунологии для решения прикладных задач. Кроме оптимизационных задач ИИС широко используется для построения многоагентных и самоорганизующихся систем моделей коллективного интеллекта и автономных распределённых систем моделей искусственной жизни и т.д.

Биологическим прототипом ИИС является иммунная система человека. Основная задача иммунной системы заключается в распознавании клеток, классификации их на своих и чужих, а также в уничтожении чужеродных клеток. С более общей точки зрения, задача иммунной системы — это уничтожение любого чужеродного агента. Агенты, воспринимаемые иммунной системой как чужеродные, называются антигенами. То есть антигены — это агенты, в ответ

на появление которых в организме иммунная система образует специальные реагирующие с ними антитела.

При попадании антитела в организм формируется ответ. Формируются первичный иммунный ответ и вторичный иммунный ответ на антигены, уже известные системе.

Первичный иммунный ответ формируется по схеме:

1. Создаётся антитело в ответ на вторжение новых антигенов
2. Адаптация антител
3. Клонирование наилучших

Вторичный иммунный ответ использует иммунную память. То есть наилучшие антитела, выработанные ранее при первом контакте с чужеродными клетками. Для использования в информационных технологиях наибольший интерес представляют функции, реализуемые посредством лимфоцитов. Лимфоциты — главные клетки иммунной системы, вырабатывающие антитела в ответ на вторжение антигенов. По функциональным признакам различают несколько типов лимфоцитов, главными из которых являются В-лимфоциты, осуществляющие распознавание антигенов и выработку антител, живущие относительно долго и хранящие в себе информацию о чужеродных клетках. При распознавании антигенов совпадение образа антигена с антителом может быть неполным. Мера близости антигена и антитела носит название «аффинность». Известно несколько теорий, объясняющих механизм производства антител иммунной системы. Наиболее известной является клонально-селекционная теория, в соответствии с которой при распознавании биклетками антигенов они стимулируются и начинают синтезировать антитела с той же специфичностью, то есть предназначенной для борьбы с данным антигеном путём клонирования. При этом число клонов пропорционально уровню стимуляции билимфоцита. Процесс, который вызывает клонирование только тех клеток, которые синтезируют нужный тип антител, называют клональным отбором. Можно сказать, что клональный отбор создаёт подпопуляцию биклеток, предназначенных для борьбы с соответствующим антигеном. После подавления антигена большая часть биклеток разрушается, а оставшиеся реализуют функцию иммунной памяти. Так, что последующее воздействие похожего антигена приводит к быстрой иммунной реакции. При проектировании ИИС также широко используются элементы теории идиотипической иммунной сети, согласно которой иммунный ответ. Иммунный ответ основывается не только на взаимодействии биклеток и антигенов, но и на взаимодействии биклеток между собой. С точки зрения информационных технологий, первичный иммунный ответ соответствует обучению иммунной системы распознаванию соответствующего антигена, а вторичный иммунный ответ отнесению новых антигенов к одной из известных групп.

Оптимизационные методы используют идеи ИИС, рассматривая антиген как подлежащую решению задачу, а антитела как векторы, соответствующие решению конкретной задачи. Известно значительное число математических моделей иммунной системы в целом, а также её различных компонентов. Для решения прикладных задач с помощью алгоритмов ИИС используют методы клонального отбора, иммунной сети, отрицательного отбора, библиотеки генов и другие. Рассмотрим модель оптимизации с помощью искусственной иммунной сети. Обозначим через $S^b = \{S_i^b\}$ популяцию антител, а через $S^g = \{S_i^g\}$ популяцию антигенов. Текущие координаты антител и антигенов обозначим через $X_i^b = \{X_{ik}^b\}$ и $X_i^g = \{X_{ij}^g\}$ соответственно. Аффинность определяется как расстояние (метрику) p между соответствующими векторами, при этом различают b-g аффинность и b-b аффинность, то есть аффинность двух антител. Чем меньше p , тем больше аффинность связей между клетками. Совокупность всех расстояний образует множества R^{bb} и R^{bg} . В случае бинарного кодирования в качестве меры близости ис-

пользуется расстояние Хэмминга. В случае вещественного кодирования — Евклидова норма.

Таким образом, иммунную сеть можно представить в виде кортежа $\langle Sb, Sg, Rbb, Rbg, Sm, nb, pc, bs, bb, br, bp \rangle$. Sm — множество антител, являющихся клетками памяти иммунной сети и данной множество является подмножеством Sb . nb — число лучших антител, отбираемых для клонирования и мутаций из Sb . pc — число клонов, создаваемых в каждом из отобранных антител. bs — степень селекции, то есть относительное число улучшенных антител, отбираемых из множества клонированных клеток. bb — пороговый коэффициент гибели. br — пороговый коэффициент сжатия. bp — коэффициент обновления иммунной сети.

Рассмотрим в этих терминах алгоритм работы ИИС:

1. Инициализация популяций Sb и Sg .
2. Для каждого из антигенов Sgi выполняем следующие действия:
 - 2.1. Вычисляем bg -аффинность антигена и Sgi со всеми антителами сети Sbj . И отбираем среди указанных антител nb агентов с максимальной аффинностью.
 - 2.2. Клонирование отобранных антител с коэффициентом клонирования, пропорциональным аффинности так, чтобы общее число клонов равнялось произведению nb на pc .
 - 2.3. Вычисляем для полученных антител их аффинность с антигеном и выбираем n антител, имеющих максимальную аффинность. Помещаем их в множество клеток памяти. $n = bs * nb * pc$. Удаляем из популяции Sm клеток памяти те клетки, аффинность которых ниже пороговой величины bb .
 - 2.4. Вычисляем аффинность всех клеток скорректированной популяции Sm и удаляем из популяции тех, аффинность которых ниже величины порога br . Расширяем популяцию Sb агентами популяции Sm .
3. Вычисляем аффинность агентов расширенной популяции и удаляем тех, величина аффинности которых ниже порога br .
4. Заменяем bp процентом худших новыми случайно сгенерированными клетками.
5. Проверяем условие останова. В зависимости от результата либо возврат к шагу 2, иначе — выход из алгоритма.

Базовый алгоритм иммунной оптимизации — это частный случай алгоритма функционирования иммунной сети в предположении что имеется один антиген — искомое значение целевой функции и bg -аффинность антитела к этому антигену равна соответствующему значению $fitness$ -функции.

Алгоритм бактериальной оптимизации

Поведение бактерий является источником новых подходов к разработке популяционных алгоритмов поисковой оптимизации. Общее название таких алгоритмов — алгоритмы бактериальной оптимизации. Известны примеры использования таких алгоритмов для решения прикладных задач из различных предметных областей. Так, они эффективно применяются для решения задач оптимального управления динамической системой или обучение нейросети. Первый алгоритм бактериальной оптимизации был предложен в 2002 году. Рассмотрим биологические предпосылки данного метода.

- Бактерия совершает движение, которое называется плаванием.
- Разворачивание бактерии называется кувырок.
- Общее поведение бактерии обусловлено механизмом, который называется хемотаксис.

- Хемотаксис — двигательная реакция микроорганизма на химический раздражитель. Данный механизм позволяет бактерии двигаться в направлении аттрактантов (питательных веществ) в противоположном направлении от репеллентов (вредных для бактерии веществ).

Если у выбранной бактерии направление движения соответствует увеличению концентрации аттрактанта, то время до следующего кувырка увеличивается. В результате бактерия совершает движение к полезным веществам от вредных. А продолжение движения в нужном направлении увеличивается. Таким образом, бактериальный хемотаксис можно определить как сложное сочетание плавания и кувырков, позволяющее бактерии удерживаться в местах высокой концентрации питательных веществ и избегать неприемлемой концентрации вредных веществ. В контексте задачи поисковой оптимизации хемотаксис можно интерпретировать как механизм оптимизации использования бактерии известных пищевых ресурсов и поиска новых потенциально более ценных.

Рассмотрим канонический алгоритм бактериальной оптимизации. Будем полагать, что значение fitness-функции F будет максимизировано. Канонический алгоритм бактериальной оптимизации (BFO — bacterial forwarding optimization) основан на трёх основных механизмах: хемотаксис, репродукция, ликвидация рассеивания. Пусть $X_{irl}(t)$ — вектор текущего положения бактерии S_i (belongs to S) на шаге t хемотаксиса, r -ом шаге репродукции и l -том шаге ликвидации рассеивания. t : от 1 до $t_{\sim}$, r : от 1 до $t_{\sim}r$, l : от 1 до $t_{\sim}l$. Число бактерий всегда чётно. t с тильдами — это общее число шагов хемотаксиса, репродукции и ликвидации рассеивания соответственно.

1) Хемотаксис — процедура хемотаксиса реализует локальную оптимизацию алгоритма. Следующее положение частицы S_i , то есть положение на шаге $t+1$ определяет формула:

$$X'_{irl} = X_{irl} + \lambda * V_i / \| V_i \|_E$$

V_i — текущий направляющий вектор шага хемотаксиса. λ — текущая величина этого шага. $\| V_i \|_E$ — Евклидова норма вектора.

При плавании бактерии вектор V_i неизменен, то есть $V'_i = V_i$. При кувырке вектор V'_i есть случайный вектор из $U_{|X|}(-1;1)$. Плавание каждой из бактерий продолжается пока происходит увеличение значений fitness-функции. Величина шага λ может меняться в процесс поиска, уменьшаясь с ростом числа итераций.

2) Репродукция — имеет своей целью ускорение сходимости. Каждой бактерии присвоим состояние здоровья h . Будем вычислять состояние здоровья так:

$$h_i = \sum_{\tau = 1}^t (F_{irl}(\tau)), i \text{ from } 1 \text{ to } |S|$$

Текущее состояние здоровья — сумма текущих значений F . Сортируем бактерии в порядке убывания состояния их здоровья и представляем результат в виде линейного списка. Механизм репродукции состоит в том, что на $r+1$ -ом шаге репродукции наиболее слабая половина бактерий исключается из списка, то есть погибает. А каждый из агентов первой половины расщепляется на две одинаковых с одинаковыми координатами. В результате этой репродукции число бактерий остаётся неизменным.

3) Ликвидация рассеивания — в общем случае рассмотренных процедур хемотаксиса и репродукции недостаточно для отыскания глобального оптимума, поскольку данные процедуры не позволяют бактериям покидать локальные оптимумы. Механизм ликвидации рассеивания

призван преодолеть этот недостаток. Данный механизм включается после выполнения определённого числа процедур репродукции и состоит в следующем: с заданной вероятностью $ksie$ случайным образом выбираем n бактерий ($n < |S|$) и вместо каждой из уничтоженных бактерий случайным образом создаём новую с тем же номером в произвольной точке в области поиска. Число бактерий в колонии в результате выполнения операции рассеивания остаётся неизменным. Запишем более строго общую схему процедуры ликвидации рассеивания:

1. Полагаем $j = 1$
2. Генерируем натуральное случайное число от 1 до S , а также вещественное число в интервале $(0;1)$
3. Если $uj > ksie$, то новые координаты бактерий определяем так: $X_{ijrlk} = U_1(xk-, xk+)$, где $xk-$ и $xk+$ являются минимальной и максимальной координатами области поиска
4. $j += 1$, пока не будет уничтожено n бактерий

Инициализация популяции, то есть определение начальных положений вектора X_{i00} осуществляется по схеме, аналогичной третьему шагу ликвидации рассеивания. Свободными являются следующие параметры: $t\sim$, $t\sim l$, $t\sim r$. λ — свободный параметр алгоритма. $ksie$ — свободный параметр. n — также свободный параметр. Диапазон, определяющий координаты бактерий — свободный параметр. Координаты бактерий, создаваемые в процессе ликвидации рассеивания, — свободные параметры.

Гибридизация популяционных алгоритмов

Опыт решения сложных прикладных задач глобальной оптимизации показывает, что применение одного алгоритма оптимизации далеко не всегда приводит к успеху. В гибридных комбинированных алгоритмах объединяющие различные (гетерогенные) либо одинаковые (гомогенные) алгоритмы, но с разными значениями параметров, преимущество одного алгоритма могут компенсировать недостатки другого. Поэтому гибридизация популяционных алгоритмов является одним из основных путей к повышению эффективности решения оптимизационных задач. Методы комбинирования алгоритмов часто называют гиперэвристиками.

Общие принципы гибридизации

Можно предложить значительное число способов гибридизации поисковых алгоритмов в общем и популяционных в частности. Рассмотрим три различных классификации:

1. Одноуровневая классификация Ванга.
2. Двухуровневая классификация Эль-Абда и Камела.
3. Четырёхуровневая классификация Рэйдла.

1. В соответствии с этой классификацией выделяют три категории гибридных алгоритмов:

- 1.1. Вложенные алгоритмы
- 1.2. Алгоритмы типа препроцессор-постпроцессор.
- 1.3. Коалгоритмы.

В категории методов гибридизации вложения выделяют высокоуровневую и низкоуровневую оптимизацию. Высокоуровневая гибридизация предполагает слабую связь объединяемых алгоритмов. Обычно основывается на комбинировании популяционного алгоритма глобального поиска и некоторого алгоритма локального поиска (необязательно популяционного). Отличии-

тельным признаком высокоуровневой гибридации является то, что объединяемые алгоритмы в гибриде сохраняют значительную автономность, то есть относительно легко можно выделить каждый из используемых алгоритмов. При низкоуровневой гибридации комбинируемые алгоритмы интегрированы настолько, что выделить составляющую итогового алгоритма практически невозможно.

Гибридация по типу препроцессор-постпроцессор является самой распространённой. Обычно используют два алгоритма, функционирующих последовательно (можно использовать и более двух алгоритмов). Выделяют два класса методов гибридации типа препроцессор-постпроцессор: последовательная предполагает фиксированный порядок использования объединяемых алгоритмов; конвейерная гибридация — более двух алгоритмов исполняются последовательно, а очерёдность включения в работу того или иного алгоритма определяется с помощью адаптивной стратегии.

Коалгоритмическая гибридация предполагает, что объединяемые алгоритмы как минимум на логическом уровне выполняются параллельно, например, обмениваясь решениями.

Отметим, что классификация Ванга не различает гибридные алгоритмы по порядку (то есть последовательному и параллельному выполнению входящих в них алгоритмов).

2. По данной классификации алгоритмы можно разделить на гомогенные и гетерогенные. Алгоритмы из данных категорий могут выполняться как последовательно, так и параллельно. Также алгоритмы могут подразделяться по способу декомпозиции пространства поиска: неявная декомпозиция, явная и гибридная (комбинирует явную и неявную). В случае последовательного поиска алгоритмы (гомогенные, то есть одни и те же алгоритмы, но с разными гиперпараметрами) функционируют последовательно так, что результат поиска предыдущего алгоритма становятся исходными данными для последующего. В случае параллельного поиска каждый из алгоритмов реализует все стадии поиска, оптимизируя свои значения за счёт эпизодического получения информации от других алгоритмов. Неявная декомпозиция основана на том, что каждый из алгоритмов ищет решение в своей подобласти пространства решений, но эти подобласти не заданы в явном виде. Явная декомпозиция — подобласти задаются явно.

3. Классификация основана на следующих признаках: тип гибридируемых алгоритмов, уровень гибридации, порядок выполнения и стратегия управления. Рассмотрим более подробно каждый из перечисленных признаков. По типу гибридируемых алгоритмов: многоагентные алгоритмы оптимизации, многоагентные алгоритмы в сочетании с проблемно-ориентированными и многоагентные алгоритмы в сочетании с алгоритмами теории исследования операций или алгоритмами ИИ (подразделяются на точные алгоритмы и алгоритмы «мягких» вычислений — алгоритмы, основанные на нечёткой логике). Уровень гибридации: высокоуровневая и низкоуровневая гибридация. Порядок выполнения комбинируемых алгоритмов: последовательный, параллельный и чередующийся. Стратегия выполнения: интегративный и кооперативный способы управления (кооперативные подразделяются на гомогенные и гетерогенные). При использовании интегративной стратегии один из алгоритмов считается зависимым или вложенным компонентом другого. Кооперативная стратегия предполагает, что гибридируемые алгоритмы обмениваются информацией, но не являются частью друг друга (островная модель параллелизма).

Вложенные алгоритмы

Высокоуровневая гибридизация вложения. Выделяют два класса высокоуровневой гибридизации: последовательная и разделения пространства поиска.

Общая схема:

1. Инициализация агентов популяционного алгоритма.
2. Выполнение фиксированного числа итераций алгоритма (обычно одной бывает достаточно).
3. Исходя из текущих положений координат осуществление локального поиска с помощью гибридизируемого алгоритма. Найденные координаты принимаем равными лучшим найденным решением.
4. Если выполнены условия окончания, то алгоритм заканчивает работу, иначе — возвращается к шагу 2.

В рассматриваемой схеме основная задача первого алгоритма обеспечить исследование пространства поиска. Целесообразно использовать модификацию алгоритма, обеспечивающую принудительное удаление близких друг к другу агентов популяции. Для этого близких агентов объединяют в кластеры и приспособленность кластеризованных агентов снижают путём деления их приспособленности на число агентов в кластере, что стимулирует удаление решений друг от друга и повышает относительную приспособленность агентов, не входящих в кластер. В качестве недостатка такого подхода можно выделить появление ещё одного свободного параметра (диаметр кластера). С точки зрения распараллеливания построенный по такой схеме алгоритм может быть реализован путём решения каждой из задач локальной оптимизации на своём процессоре.

Высокоуровневая гибридизация вложения на основе декомпозиции пространства поиска. Обычно пространство для поиска раскладывается на несколько подпространств: в случае двух алгоритмов — на два подпространства — $|X|/2$. Размерность каждого подпространства равно двум. Для простоты записи примем, что $|X|$ — чётное число, тогда последовательность шагов примет вид:

1. Инициализация мультипопуляции. $S = \{S_i, i \text{ belongs to } [1; |X|/2]\}$, $|X|/2 = |S|$
2. Выполняем с помощью популяции S_1 поиск новых компонент x_1 и x_2 вектора x , используя в качестве остальных компонент их значения, полученные на предыдущей итерации.
3. Выполняем с помощью популяции S_2 поиск новых значений компонент x_3 и x_4 вектора x , используя при этом в качестве компонент x_1 и x_2 значения этих величин, полученные на шаге 2. В качестве остальных компонент — их значения, полученные на предыдущей итерации и т.д.

...

Шаг $|X|/2$. Выполняем с помощью популяции $S_{|S|}$ поиск новых значений компонент $x_{|x|-1}$ и $x_{|x|}$, используя в качестве значений $x_1, x_2, \dots, x_{|x|-2}$ их значения, найденные на предыдущих шагах.

Последний шаг. Проверка условий окончания поиска — в случае если они достигнут в качестве решения, принимаем решение, полученное на предыдущем шаге, а иначе — возврат к шагу 2.

Низкоуровневая гибридизация вложением. Низкоуровневая гибридизация предполагает сращивание гибридизируемых алгоритмов так, что её результатом является новый алгоритм, который нельзя разделить на составляющие. Общей методики такой гибридизации нет. Единственная рекомендация — модификация миграционных операторов первого алгоритма на основе второго (гибридизируемого) алгоритма. Приведём несколько примеров такой гибридизации: ГА + локальная оптимизация (например, градиентный спуск) — низкоуровневая гибридизация в таком случае может состоять в замене операции мутации несколькими шагами локальной оптимизации; БО + ПСО — низкоуровневая оптимизация здесь возможна путём реализации хемотаксиса с помощью одной или нескольких итераций алгоритма роя частиц; ПСО + ГА — при гибридизации этих алгоритмов каждой частице роя можно поставить в соответствие хромосому, число генов которой равно размерности вектора координат частиц, — хромосомы в таком гибриде эволюционируют как по правилам движения частиц роя, так и по правилам ГА.

Гибридизация препроцессор-постпроцессор. Рассмотрим критерии, влияющие на эффективность последовательной гибридизации. Выбор гибридизируемых алгоритмов: для последовательной гибридизации, как правило, выбирают популяционный алгоритм + локальную оптимизацию. Интерфейс между алгоритмами: так как при большом числе итераций все популяционные алгоритмы, как правило, сходятся к глобальному экстремуму, то в качестве стартовой точки для локального алгоритма, кажется что, достаточно использовать лучшую точку глобального, однако в связи с тем, что локальный алгоритм предполагает достаточно ранее прерывания, выбор одной точки может привести к потере глобального экстремума, поэтому в качестве стартовой следует передавать не только лучшую точку, но и некоторое число других точек — общее число таких точек обозначим через n_{start} . Пусть переключение алгоритмов производится после завершения t_n итераций. Существует несколько способов переключения: все точки, найденные одним алгоритмом, подаются на вход следующему; только лучшие точки, найденные одним алгоритмом, подаются на вход следующему — среди отобранных по этой схеме точек могут оказаться близкие, тогда отбираем точки так, чтобы среди отобранных не было точек, расположенных близко друг к другу ($\|x_i - x_j\| > e, i \neq j$). Так как переход от локального к глобальному поиску осуществляется достаточно рано, то можно предложить ещё одну схему отбора: выбираем n'_{start} особей по правилу близости, а остальных $n_{\text{start}} - n'_{\text{start}}$ выбираем равномерно из числа оставшихся. Ещё один метод: хранить координаты лучших точек на протяжении t_n итераций популяционного и передать их на вход следующему. Момент времени: поскольку на начальных итерациях особи широко распределены в области поиска, но далеки от решения задачи, а на завершающих итерациях особи сосредоточены в окрестности найденного экстремума (необязательно глобального), то в качестве верхней границы момента времени переключения между алгоритмами целесообразно принимать момент начала стагнации вычислительного процесса. Или же можно жёстко задать значение t_n . Закон применения гибридизируемых алгоритмов: начало работы второго алгоритма после определённого числа итераций работы первого алгоритма.

Конвейерная гибридизация. Это гибридизация более двух алгоритмов. Выбор алгоритмов: имеет смысл объединять только гомогенные алгоритмы. Реализация интерфейсов: в зависимости от принципов реализации интерфейсов алгоритмы имеет смысл разделять на два класса — нестигмергичные — не используют канал стигмергии, стигмергичные — используют канал стигмергии. Стигмергия — след.

Гибридизация нестигмергичных алгоритмов. Состояние агента, как правило, определяют его текущие координаты, а также, возможно, скорость. Поэтому реализация интерфейса между такими алгоритмами сводится к передаче координат агентов предыдущей популяции (конечных координат) в качестве начальных координат агентов новой популяции. При гибридизации стигмергичных алгоритмов метки в пространстве поиска, которые оставляют агенты, формируют модель fitness-функции решаемой задачи поисковой оптимизации. Поэтому главная проблема организации интерфейса между такими алгоритмами — это преобразование этих моделей. Существует возможность гибридизации стигмергичных и нестигмергичных алгоритмов. Если алгоритмы относятся к разным классам, то их комбинирование приводит к потере информации о модели fitness-функции. Например, если объединить алгоритм колонии муравьёв и роя частиц, то новых агентов роевого алгоритма целесообразно инициализировать в областях с высокой интенсивностью феромонного следа.

В простейшем случае момент переключения с одного алгоритма на другой задаётся пользователем: $[0;t_1), [t_1;t_2), \dots$. Более сложным является вариант адаптивного управления переключения алгоритмов. При этом используются следующие условия: стагнация вычислительного процесса, замедление скорости сходимости ниже допустимого порога, комбинация этих двух условий.

Закон переключения алгоритмов. Общая теория адаптивной конвейерной гибридизации предложена в работах Растригина. Рассмотрим основы этой теории.

Пусть имеется совокупность алгоритмов поисковой оптимизации $A = \{a_1, \dots, a_{|A|}\}$. С помощью каждого из этих алгоритмов может быть решена задача глобальной оптимизации. Без потери общности рассуждений примем, что в процессе решения задачи в течение итерации $[t_{n-1}; t_n)$ работает алгоритм a_n . К моменту итерации t_n выполнены испытания в некоторой совокупности точек x_n , вычислены значения функции в этих точках, а также имеются множества ограничивающих функций. Тогда в общей постановке задачи альтернативная адаптация состоит в отыскании алгоритма $a_{n+1} = R(X_n, f_n, E_n, G, A_n, W_n)$ на $[t_n; t_{n+1})$. $A_n = \{a_k \text{ belongs to } A, k \text{ belongs to } [1, n]\}$. W_n — совокупность оценок критерия качества адаптации.

Коалгоритмы

Коалгоритмы можно интерпретировать как алгоритмы эволюционирующих систем. Коэволюция — ситуация, когда в процессе эволюции система А влияет на систему В, а В приспосабливается к изменениям А и наоборот.

Классификация коэволюционных алгоритмов. Наиболее известная модель — островная модель параллелизма. Её суть состоит в том, что эволюционирующую популяцию делят на субпопуляции, которые эволюционируют, по крайней мере на логическом уровне, параллельно. Эпизодически некоторые агенты перемещаются из одной субпопуляции в другую, передавая свой опыт. Коэволюционные алгоритмы классифицируют по: используемой модели коэволюции, форме коэволюции, числу коэволюционирующих популяций и по схеме взаимодействия между ними.

Модель коэволюции. Выделяют два класса вычислительных моделей коэволюции: простые модели (в них в качестве решения исходной задачи может быть использовано решение,

найденное любой из коэволюционирующих субпопуляций) и композиционные модели (тут решение задачи есть объединение фрагментов, найденных различными субпопуляциями).

Форма коэволюции. В зависимости от характера взаимодействия между субпопуляциями различают две формы коэволюции: сотрудничество и соперничество. Сотрудничество предполагает, что каждая субпопуляция решает либо одну задачу, либо её часть. Важным случаем является композиционно-кооперативное сотрудничество, когда каждая из субпопуляций отыскивает свою часть общего решения задачи, при этом может использоваться либо декомпозиция вектора варьируемых параметров, либо декомпозиция целевой функции. Коэволюция данного типа целесообразна в случае решения задач высокой вычислительной или структурной сложности. Соперничество использует один из следующих типов взаимодействия: взаимодействие master-slave, хозяин-подчинённый, хозяин-паразит (перераспределение ресурсов между субпопуляциями отсутствует); взаимодействие субпопуляций, имеющих различные области поиска; взаимодействие популяций, отличающихся стратегиями поиска.

Число коэволюционирующих субпопуляций. Здесь различают многопопуляционные коэволюции и, как частный случай, двухпопуляционные.

Взаимодействие между субпопуляциями. Выделяют взаимодействие в последовательной и параллельной схемах. В последовательной схеме обновление текущего числа агентов производится последовательно так, что текущая численность данной субпопуляции зависит от её численности на предыдущей итерации и текущей численности уже обновлённой субпопуляции. В параллельной схеме обновление осуществляется параллельно. Возможны различные комбинации последовательной и параллельной схем.

Методы распараллеливания популяционных алгоритмов оптимизации.

Глобальная модель параллелизма. Популяционные алгоритмы, построенные на основе этой модели, представляют собой параллельные аналоги соответствующих последовательных алгоритмов. Данные алгоритмы используют схему параллелизма по данным и ориентированы на организацию параллельных вычислений по схеме master-slave. Мастер-процесс в этом случае выполняется на хост-процессоре параллельной ВМ и реализует сам популяционный алгоритм. Каждый из подчинённых (или рабочих) процессов назначают на выполнение одному из процессоров. Подчинённый (или рабочий) процесс производит вычисления значений fitness-функции, а также фазовых координат соответствующего агента. Функции рабочих процессов могут ограничиваться только вычислением значений fitness-функции. Рабочий процесс после каждой итерации отправляет мастер-процессу соответствующее значение fitness-функции, а мастер-процесс, в свою очередь, на основе этих данных вычисляет новые фазовые координаты агентов и отправляет их рабочим процессам. Параллельные вычисления в алгоритмах рассматриваемого класса могут быть как синхронными, так и асинхронными. Глобальная синхронная модель параллелизма предполагает, что текущая итерация совершается только после того, как мастер-процессором полученных значений fitness-функции от всех рабочих процессоров. Глобальная синхронная модель предполагает использование так называемой барьерной синхронизации. Вычисления мастер-процессора характеристик популяции производится только после получения этим процессором указанной информации обо всех агентах популяции. Условиями высокой производительности алгоритма, использующего данную модель, является гомогенность вычислений и одинаковое время вычисления значений fitness-функции в любой допустимой точке пространства параметров. Как правило, обеспечить выполнение всех этих условий не удаётся, так как зачастую приходится использовать гетерогенные вы-

числительные системы, а в реальных оптимизационных задачах вычислительная сложность fitness-функции может зависеть от значений компонент вектора варьируемых параметров.

Вторая модель носит название глобальная асинхронная модель параллелизма. Исходит из того, что мастер-процессор получает данные от рабочих процессов не после глобальной синхронизации, а в любой момент времени по мере готовности этих данных. На основе полученной информации мастер-процессор обновляет скорости и координаты соответствующих агентов популяции и немедленно возвращает их свободно рабочим процессорам для продолжения итераций. В алгоритме мастер-процесс может работать с подчинёнными по схеме FIFO. То есть обновлять фазовые координаты первого агента в очереди и посылать их первому свободному процессу. Достоинством параллельных алгоритмов, основанных на глобальной модели, является возможность получения и использования глобальной информации о популяции. Результат оптимизации, полученный при использовании глобальной синхронной модели, является таким же, как при аналогичных последовательных вычислениях. Недостатком алгоритмов данного класса является высокие накладные расходы на коммуникацию.

Островная модель параллелизма. В некоторых публикациях также называют миграционной, а алгоритмы, построенные на ней, называют миграционными или аккумулирующими мультипопуляции. Суть данной модели в следующем: пусть имеется мультипопуляция $S = \bigcup S_i$, S_i — остров, число островов соответствует числу процессоров. Число агентов мультипопуляции есть сумма агентов всех субпопуляций. Каждый остров обрабатывает свой процессор системы. Обмен данными между островами происходит каждые $tmig$ независимых итераций в соответствии с используемой топологией соседства островов. Самая распространённая топология — кольцо. Период, состоящий из $tmig$ операций, в течение которого субпопуляции развиваются независимо, называют сезоном. Островная модель, как правило, обеспечивает высокую производительность при выполнении следующих условий: гомогенность, размеры субпопуляций должны быть одинаковы и достаточно велики, условием останова для каждой субпопуляции является достижение одного и того же предельного числа итераций $t_{\sim}$. Так как популяционные алгоритмы являются стохастическими, при разных запусках популяция может сходиться к разным локальным решениям. Островная модель позволяет запустить алгоритм сразу несколько раз и совместить достижения разных островов для получения наилучшего. С другой стороны, это яркий пример многопопуляционного алгоритма. Известно большое число популяций островной модели, определяющееся топологией соседства островов, длительностью сезона ($tmig$), стратегией миграции и режимом обмена между агентами. При больших значениях $tmig$ расходы на коммуникацию минимальны, но недостаточно эффективно используется о других островах. Существуют варианты задать динамическую длительность сезона, когда правила изменения величины сезона могут быть адаптивными или неадаптивными. В первом случае $tmig$ представляет собой функцию от числа итераций, а во втором — определяется текущим состоянием острова. Наиболее известны две стратегии: миграция перемещением (заключается в замене лучшего агента данного острова худшим агентом другого острова) и миграция репликацией (в каждом острове осуществляется поиск лучшего и худшего агентов, а далее из всех особей выбирают наилучшего и осуществляют репликацию — копирование — во все остальные острова на место наихудшего: основная цель — интенсификация). Режим обмена: различают синхронную модель (когда все острова ожидают завершения обработки своих данных, после чего осуществляется обмен) и асинхронную модель (обмен данными осуществляется по мере их готовности).